

OrderBook Design

Architect / Modules

main.cpp

- the main function

feed_handler.cpp

- message entrypoint

message_parser.cpp

- parse each input

order_book.cpp

- orderbook implementation

orderbook_types.h

- most of the data structure definition

error_monitor.h

- error tracker

Data Structure

The OrderBook

Use `unordered_map` to track each product and its orderbook (buy and sell)

see:

```
class PriceInfoList :public std::list<PriceInfo> {
private:
    TradeSize totalTradeSize_;
public:
    uint32_t getTotalTradeSize() const{
        return totalTradeSize_;
    }
    void changeTradeSize(TradeSize sz) {
        totalTradeSize_+=sz;
    }
};

typedef std::map<Price, PriceInfoList> PriceOrderBook_t;
typedef std::unordered_map<ProductId,
std::pair<PriceOrderBook_t,PriceOrderBook_t>> ProductPriceMap_t;
```

also use `unordered_map` to track all the orders

```
typedef std::unordered_map<OrderId, Order> AllOrders_t;
```

Other approaches of data structure

I have considered a `vector` as order book data structure.

PROS:

- cache friendly, `vector` memory layout is continuous.
- $O(1)$ complexity when access

CONS:

- might be sparse and need more space. but in realworld it shouldn't be a problem, most order will be exist in a very narrow price range.
- the worst case could be $O(N)$ when update Minimum ask price and maximum bid price.
- iterate order book is not convenient and could be $O(N)$ complexity when orders distribute in a big range.

I choosed `std::map` for easy to implementation.

- Add order : $\log N$
- Modify : $\log N \sim 2\log N$ (remove and add again)
- Remove : $\log N$
- Trade : $2\log N \sim 2 O(N)$ (if all orders are same price, map will degrade to list)

Some assumptions

`common_def.h`

```
namespace ob {  
    const size_t MAX_TRADE_SIZE = 1000000;  
    const size_t MAX_TRADE_PRICE = 10000000;  
  
    const size_t MAX_MSG_LENGTH = 500;  
    const size_t MIN_MSG_LENGTH = 5;  
}
```

Bid/ask price cross check

when best sell order price at or below best buy order price but no trades occur.

If there is a missing trade message, the orderbook data could be incorrect, and probably process the following messages incorrectly. Unless we implement a match engine to fix it.

Modify order

allow to modify order side, price and quantity

- change side or price, will remove the old order, then add new one
- change quantity, will modify on the existing order, keep the origin priority.

Build & Run

Env

docker (ubuntu 18)

```
g++ -v
Using built-in specs.
COLLECT_GCC=g++
COLLECT_LTO_WRAPPER=/usr/lib/gcc/x86_64-linux-gnu/7/lto-wrapper
OFFLOAD_TARGET_NAMES=nvptx-none
OFFLOAD_TARGET_DEFAULT=1
Target: x86_64-linux-gnu
Configured with: ../src/configure -v --with-pkgversion='Ubuntu 7.5.0-3ubuntu1~18.04' --with-bugurl=file:///usr/share/doc/gcc-7/README.Bugs --enable-languages=c,ada,c++,go,brig,d,fortran,objc,obj-c++ --prefix=/usr --with-gcc-major-version-only --program-suffix=-7 --program-prefix=x86_64-linux-gnu- --enable-shared --enable-linker-build-id --libexecdir=/usr/lib --without-included-gettext --enable-threads=posix --libdir=/usr/lib --enable-nls --enable-bootstrap --enable-clocale=gnu --enable-libstdcxx-debug --enable-libstdcxx-time=yes --with-default-libstdcxx-abi=new --enable-gnu-unique-object --disable-vtable-verify --enable-libmpx --enable-plugin --enable-default-pie --with-system-zlib --with-target-system-zlib --enable-objc-gc=auto --enable-multiarch --disable-werror --with-arch=32=i686 --with-abi=m64 --with-multilib-list=m32,m64,mx32 --enable-multilib --with-tune=generic --enable-offload-targets=nvptx-none --without-cuda-driver --enable-checking=release --build=x86_64-linux-gnu --host=x86_64-linux-gnu --target=x86_64-linux-gnu
Thread model: posix
gcc version 7.5.0 (Ubuntu 7.5.0-3ubuntu1~18.04)
```

native macos

```
Configured with: --prefix=/Applications/Xcode.app/Contents/Developer/usr --with-gxx-include-dir=/usr/include/c++/4.2.1
Apple LLVM version 8.1.0 (clang-802.0.42)
Target: x86_64-apple-darwin16.7.0
Thread model: posix
InstalledDir:
```

```
/Applications/Xcode.app/Contents/Developer/Toolchains/XcodeDefault.xctoolchain/usr/bin
```

build

```
mkdir build && cd build
cmake ../
make
make install
```

or build with docker env (it need some time to build docker first)

```
make
```

run

```
bin/orderbook data/sample_msg.txt
```

```
--- ORDER BOOK -----
Product 5
    1075  S 1
    1050  S 10
    1025  S 2 S 5
    1050  B 3
    1000  B 9 B 1
    975   B 30
--- TRADE -----
product 5:2@1025
--- TRADE -----
product 5:3@1025
--- ORDER BOOK -----
Product 5
    1075  S 1
    1050  S 10
    1025  S 4
    1000  B 9 B 1
    975   B 30
--- STATS -----
                                Corrupted Message      0
                                Duplicated Add           0
                                Trade on missing order    0
                                Remove/Modify with no corresponding order 0
                                No trade occur when ask/price cross 0
                                Invalid Message(negative/invalid data) 0
                                Invalid product ID        0
                                Remove order with wrong data 0
                                Modify on a deleted order  0
```

```
total time: 49646 per msg:3309 nanoseconds.
```

Performance test

run on iMac i5-4670

```
bin/perf_test data/p1_300k.txt  
total time: 125025602 per msg:395 nanoseconds.
```

```
bin/perf_test data/p10_300k.txt  
total time: 134066917 per msg:405 nanoseconds.
```

Unit Test

I don't use any test framework. refer to

```
unittest/unittest.cpp
```

run

```
bin/unittest
```

test1

- basic message scenario described in doc.

test2

- verify it will output total quantity traded and product id

test3

- invalid message data, product id

```
N,5,100001,S,10,1000  
N,-5,100002,B,10,1000 // invalid product id  
N,5,100007,B,0,1000   // invalid quantity  
N,5,100008,B,9,1000  
N,5,100009,S,8,1900   // should be a trade message here
```

test4

- bid/ask price cross check error e.g. 1

```
N,5,100001,S,10,1000
N,5,100008,B,9,1000
X,5,9,1000           // do order matching, so that orderbook can reflect
the latest status
M,100001,S,1,1000    // bid/ask price cross check and no error found
```

test5

- bid/ask price cross check error e.g. 2

```
N,5,100001,S,10,1000
N,5,100008,B,9,1000
N,5,100009,S,8,1900  // should be a trade message here, will report error
N,5,100010,B,1,1100  // still bid/ask price cross check error
```

test6

- bid/ask price cross check error the e.g.3

```
N,5,100001,S,10,1000
N,5,100008,B,9,1000
R,100008,B,9,1000    // missing trade message, report error too.
M,100001,S,1,1000    //
```

test7

- test the match engine to generate message

use the basic message scenario of doc

```
N,5,100000,S,1,1075
N,5,100001,B,9,1000
N,5,100002,B,30,975
N,5,100003,S,10,1050
N,5,100004,B,10,950
N,5,100005,S,2,1025
N,5,100006,B,1,1000
R,100004,B,10,950
N,5,100007,S,5,1025
N,5,100008,B,3,1050
X,5,2,1025           // remove the last 5 lines
```

```
X,5,1,1025
R,100008,B,3,1050
R,100005,S,2,1025
M,100007,S,4,1025
```

to see if the match engine can generate the same/similar message flows

actual result

```
N,5,100000,S,1,1075
N,5,100001,B,9,1000
N,5,100002,B,30,975
N,5,100003,S,10,1050
N,5,100004,B,10,950
N,5,100005,S,2,1025
N,5,100006,B,1,1000
R,100004,B,10,950
N,5,100007,S,5,1025
N,5,100008,B,3,1050
X,5,2,1025
X,5,1,1025
R,100005,S,2,1025
M,100008,B,1,1025
M,100007,S,4,1025
R,100008,B,1,1025
```

At the moment I haven't merged the messages for the order '100008' ('M' and 'R') so there are 2 messages generated, but the order book is good

```
--- ORDER BOOK -----
Product 5
    1075  S 1
    1050  S 10
    1025  S 4
    1000  B 9 B 1
    975   B 30
```

Test result summary

```
=== SUMMARY =====
Basic unit test orderbook PASSED
test most recent trade PASSED
test most recent trade PASSED
test bid ask price cross PASSED
test invalid msg data PASSED
test invalid productid PASSED
test bid ask price cross(e.g.1) PASSED
```

```
test bid ask price cross(e.g.2) PASSED
test bid ask price cross(e.g.3) PASSED
message generate test PASSED
```

Message Generator

I wrote a message generator to generate a huge test data for performance test purpose. The main idea is explained below:

match engine

1. create 'N'(new) order with some random data, e.g. price, quantity, productid,
2. use orderbook API to check whether there is a match order, generate the corresponding trade, remove and modify message
3. go back to step 2 until there is no more message created.
4. go back to step 1 to generate more messages.

refer to

```
perftest/msg_generator.cpp

and order_book.cpp
void OrderBook::doOrderMatch(vector<string> &tradeVec, vector<string>
&orderVec)
```

usage

```
bin/msg_generator 1 1 10 5 80 5000 15k.txt 0
    or default setting
bin/msg_generator
```

arguments explained (with default value)

```
arg1: 1;  number of products
arg2: 1;  order quantity start from 1
arg3: 10; order quantity range = 10, means quantity will be 1~10;
arg4: 5;  price range is 5 unit
arg5: 80; price start from 80, means price will be 80~84 if range is 5
arg6: 5000; number of order, usually it will get x3 message when replay
the messages
```



```
arg7: 15k.txt; output file, default is messages.txt
arg8: 1; 1:print each message,0:no print
```

example

the result is verified

```
cmake-build-debug/msg_generator
use default setting
N,1,1,B,2,84
N,1,2,S,9,83
X,1,2,83
--- TRADE -----
product 1:2@83
M,2,S,7,83
R,1,B,2,84
N,1,3,B,3,82
N,1,4,S,8,84
N,1,5,B,4,81
N,1,6,B,10,80
N,1,7,B,9,82
N,1,8,B,6,84
X,1,6,83
--- TRADE -----
product 1:8@83
M,2,S,1,83
R,8,B,6,84
N,1,9,B,4,80
N,1,10,B,3,84
X,1,1,83
--- TRADE -----
product 1:9@83
X,1,2,84
--- TRADE -----
product 1:2@84
R,2,S,1,83
M,10,B,2,84
M,4,S,6,84
R,10,B,2,84
--- ORDER BOOK -----
Product 1
    84  S 6
    82  B 3 B 9
    81  B 4
    80  B 10 B 4
--- STATS -----
Corrupted Message 0
Duplicated Add 0
Trade on missing order 0
Remove/Modify with no corresponding order 0
No trade occur when ask/price cross 0
Invalid Message(negative/invalid data) 0
```

```
Invalid product ID      0
Remove order with wrong data 0
Modify on a deleted order 0

=== RESULT =====
generated file messages.txt
```

Test data

I have put some data in this folder data/

p1_300k.txt

- 1 product, 300k messages

p10_300k.txt

- 10 products, 300k messages

sample_msg.txt

- the basic sample message

TODO

match engine

As I described in unit test 7, it can be improved by merging the order message.

performance profile

Haven't tried with any profiling tool to analyze the performance yet.